STEP 1: Import Libraries

```
# Data handling
import pandas as pd
import numpy as np

# Text preprocessing
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Feature extraction
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

# Topic modeling
from sklearn.decomposition import LatentDirichletAllocation, NMF

# Download NLTK data
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

STEP 2: Load Dataset

```
documents = [
    "Machine learning improves model accuracy using data",
    "Deep learning uses neural networks for complex tasks",
    "Cancer treatment includes chemotherapy and radiation",
    "Healthcare systems manage patient data and diagnosis",
    "Wireless networks enable communication over distances",
    "Signal processing is important in communication systems",
    "Renewable energy sources include solar and wind power",
    "Power systems focus on energy efficiency",
] * 25   # multiply to get 200 documents

print("Total Documents:", len(documents))
```

```
Total Documents: 200
```

STEP 3: Text Preprocessing

```
nltk.download('punkt_tab')

stop_words = set(stopwords.words('english'))

def preprocess(text):
    # Lowercase
    text = text.lower()

    # Remove numbers and punctuation
    text = re.sub(r'[^a-z\s]', '', text)

    # Tokenization
    words = word_tokenize(text)

    # Remove stopwords
    words = [word for word in words if word not in stop_words]

    return " ".join(words)

# Apply preprocessing
processed_docs = [preprocess(doc) for doc in documents]

print(processed_docs[:5])
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
['machine learning improves model accuracy using data', 'deep learning uses neural networks complex tasks', 'cancer treatmer
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
```

STEP 4: Feature Representation ✅ For LDA (Bag of Words)

```
count_vectorizer = CountVectorizer()
X_count = count_vectorizer.fit_transform(processed_docs)
```

For NMF (TF-IDF)

```
tfidf_vectorizer = TfidfVectorizer()
X_tfidf = tfidf_vectorizer.fit_transform(processed_docs)
```

STEP 5: Build LDA Model

```
num_topics = 5

lda_model = LatentDirichletAllocation(n_components=num_topics, random_state=42)
lda_model.fit(X_count)
```

```
▾ LatentDirichletAllocation
LatentDirichletAllocation(n_components=5, random_state=42)
```

STEP 6: Display LDA Topics

```
def display_topics(model, feature_names, num_words):
    for i, topic in enumerate(model.components_):
        print(f"\nTopic {i+1}:")
        print([feature_names[j] for j in topic.argsort()[-num_words:]])

# Show topics
display_topics(lda_model, count_vectorizer.get_feature_names_out(), 5)
```

```
Topic 1:
['healthcare', 'manage', 'patient', 'data', 'systems']

Topic 2:
['important', 'processing', 'signal', 'communication', 'systems']

Topic 3:
['deep', 'complex', 'uses', 'tasks', 'learning']

Topic 4:
['includes', 'chemotherapy', 'treatment', 'radiation', 'networks']

Topic 5:
['renewable', 'sources', 'wind', 'energy', 'power']
```

STEP 7: Build NMF Model

```
nmf_model = NMF(n_components=num_topics, random_state=42)
nmf_model.fit(X_tfidf)
```

```
▾ NMF
NMF(n_components=5, random_state=42)
```

STEP 8: Display NMF Topics

```
display_topics(nmf_model, tfidf_vectorizer.get_feature_names_out(), 5)
```

```
Topic 1:
['wind', 'efficiency', 'focus', 'energy', 'power']

Topic 2:
['wireless', 'processing', 'important', 'signal', 'communication']

Topic 3:
['diagnosis', 'manage', 'healthcare', 'patient', 'data']

Topic 4:
['deep', 'complex', 'neural', 'tasks', 'uses']

Topic 5:
['cancer', 'chemotherapy', 'radiation', 'includes', 'treatment']
```

STEP 9: Compare LDA vs NMF

```
print("\nComparison:")
print("1. NMF gives clearer topics.")
print("2. LDA shows more word overlap.")
print("3. NMF works better for short text.")
print("4. LDA is probabilistic; NMF is matrix-based.")
print("5. NMF is more sensitive to preprocessing.")
```

```
Comparison:
1. NMF gives clearer topics.
2. LDA shows more word overlap.
3. NMF works better for short text.
4. LDA is probabilistic; NMF is matrix-based.
5. NMF is more sensitive to preprocessing.
```